

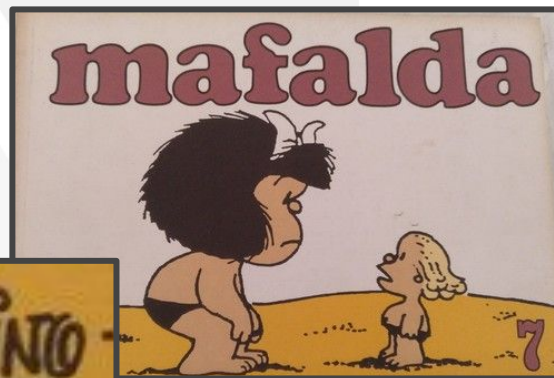


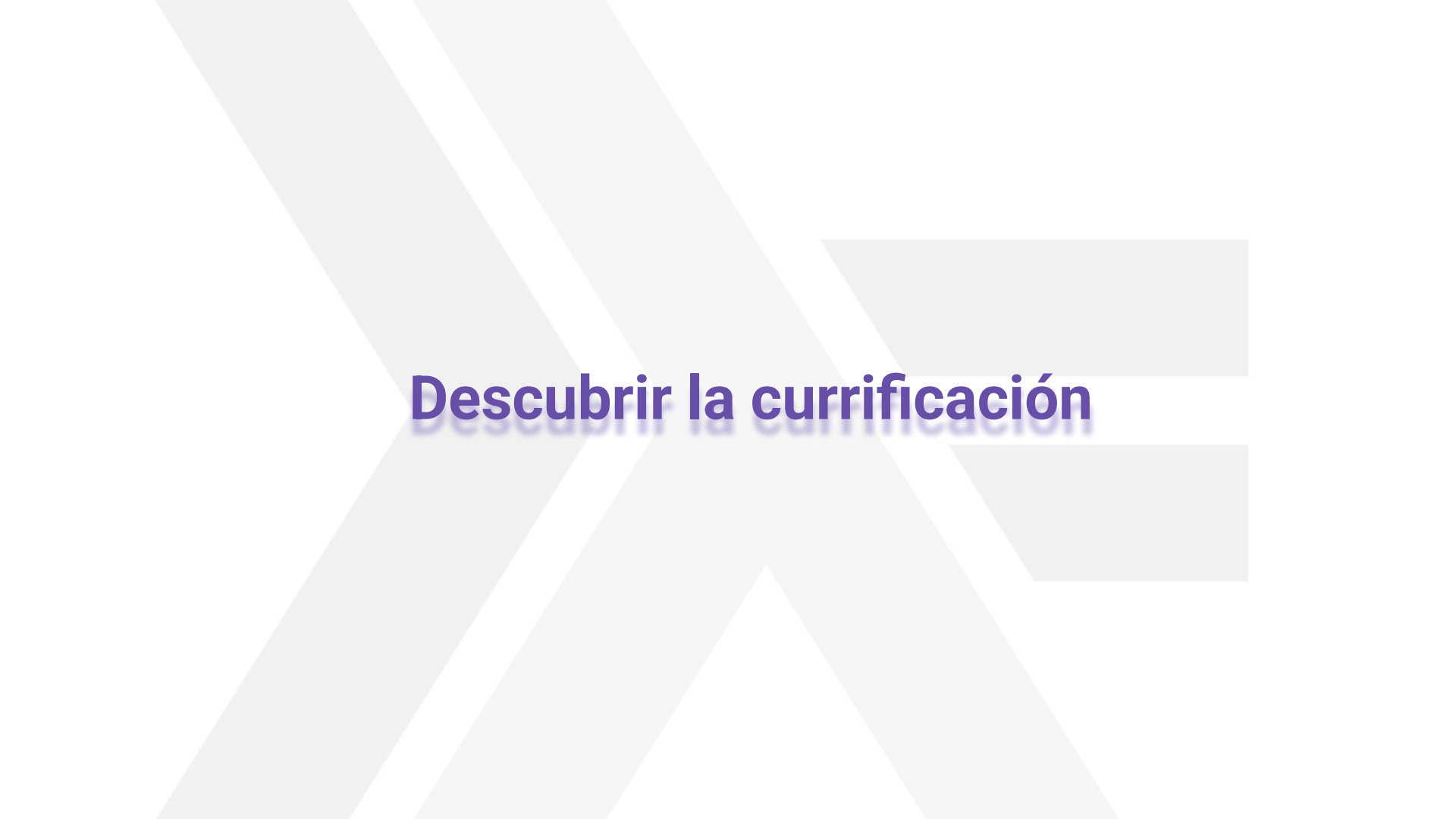
Programación Funcional

Clases teóricas

por Pablo E. “Fidel” Martínez López

3. Currificación





Descubrir la currificación

Aplicación de funciones de orden superior

- Considerar las siguientes definiciones

suma' :: ...

suma' (x,y) = x+y

suma :: ...

suma x = f where f y = x+y

- ¿Qué tipo tienen las funciones?
- ¿Qué similitudes hay entre ellas?
- ¿Y qué diferencias?

Aplicación de funciones de orden superior

- Considerar las siguientes definiciones

```
suma' :: (    ,    ) ->
```

```
suma' (x,y) = x+y
```

```
suma :: ...
```

```
suma x = f where f y = x+y
```

- ¿Qué tipo tienen las funciones?

Aplicación de funciones de orden superior

- Considerar las siguientes definiciones

```
suma' :: (    ,    ) -> Int
```

```
suma' (x,y) = x+y
```

```
suma :: ...
```

```
suma x = f where f y = x+y
```

- ¿Qué tipo tienen las funciones?

Aplicación de funciones de orden superior

- Considerar las siguientes definiciones

```
suma' :: (Int,Int) -> Int
```

```
suma' (x,y) = x+y
```

```
suma :: ...
```

```
suma x = f where f y = x+y
```

- ¿Qué tipo tienen las funciones?

Aplicación de funciones de orden superior

- Considerar las siguientes definiciones

suma' :: (Int,Int) -> Int

suma' (x,y) = x+y

suma :: ...

suma x = f where f y = x+y

Una función que toma un par de enteros y devuelve un entero

- ¿Qué tipo tienen las funciones?

Aplicación de funciones de orden superior

- Considerar las siguientes definiciones

```
suma' :: (Int,Int) -> Int
```

```
suma' (x,y) = x+y
```

```
suma :: ...
```

```
suma x = f where f y = x+y
```

- ¿Qué tipo tienen las funciones?

Aplicación de funciones de orden superior

- Considerar las siguientes definiciones

```
suma' :: (Int,Int) -> Int
```

```
suma' (x,y) = x+y
```

```
suma ::      ->
```

```
suma x = f where f y = x+y
```

- ¿Qué tipo tienen las funciones?

Aplicación de funciones de orden superior

- Considerar las siguientes definiciones

```
suma' :: (Int,Int) -> Int
```

```
suma' (x,y) = x+y
```

```
suma ::      -> (      ->      )
```

```
suma x = f where f y = x+y
```

- ¿Qué tipo tienen las funciones?

Aplicación de funciones de orden superior

- Considerar las siguientes definiciones

```
suma' :: (Int,Int) -> Int
```

```
suma' (x,y) = x+y
```

```
suma ::      -> (      -> Int)
```

```
suma x = f where f y = x+y
```

- ¿Qué tipo tienen las funciones?

Aplicación de funciones de orden superior

- Considerar las siguientes definiciones

```
suma' :: (Int,Int) -> Int
```

```
suma' (x,y) = x+y
```

```
suma ::      -> (Int -> Int)
```

```
suma x = f where f y = x+y
```

- ¿Qué tipo tienen las funciones?

Aplicación de funciones de orden superior

- Considerar las siguientes definiciones

```
suma' :: (Int,Int) -> Int
```

```
suma' (x,y) = x+y
```

```
suma :: Int -> (Int -> Int)
```

```
suma x = f where f y = x+y
```

- ¿Qué tipo tienen las funciones?

Aplicación de funciones de orden superior

- Considerar las siguientes definiciones

```
suma' :: (Int,Int) -> Int
```

```
suma' (x,y) = x+y
```

```
suma :: Int -> (Int -> Int)
```

```
suma x = f where f y = x+y
```

- ¿Qué tipo tienen las funciones?

Una función que toma
un entero y devuelve
una función de enteros
en enteros

Aplicación de funciones de orden superior

- ❑ Considerar las siguientes definiciones

suma' :: (Int,Int) -> Int

suma' (x,y) = x+y

Una función que toma un par de enteros y devuelve un entero

suma :: Int -> (Int -> Int)

suma x = f where f y = x+y

Una función que toma un entero y devuelve una función de enteros en enteros

- ❑ ¿Qué tipo tienen las funciones?
- ❑ ¿Qué similitudes hay entre ellas?
- ❑ ¿Y qué diferencias?

Aplicación de funciones de orden superior

- Similitudes
 - Ambas expresan la suma de dos enteros
 - ¡Pero no de la misma manera!

Aplicación de funciones de orden

```
suma' :: (Int,Int) -> Int
suma' (x,y) = x+y

suma :: Int -> (Int -> Int)
suma x = f where f y = x+y
```

■ Similitudes

■ Ambas expresan la suma de dos enteros

■ ¡Pero no de la misma manera!

■ para todos x e y , $\text{suma}' (x,y) = (\text{suma } x) y$

Aplicación de funciones de orden

```
suma' :: (Int,Int) -> Int
suma' (x,y) = x+y

suma :: Int -> (Int -> Int)
suma x = f where f y = x+y
```

■ Similitudes

- Ambas expresan la suma de dos enteros
- ¡Pero no de la misma manera!
 - para todos x e y , $\text{suma}' (x,y) = (\text{suma } x) y$

■ Diferencias

- Una toma un par, la otra toma un número
- Una retorna un número, la otra retorna *una función*

Aplicación de funciones de orden

```
suma' :: (Int,Int) -> Int
suma' (x,y) = x+y

suma :: Int -> (Int -> Int)
suma x = f where f y = x+y
```

Similitudes

- Ambas expresan la suma de dos enteros
- ¡Pero no de la misma manera!
 - para todos x e y , $\text{suma}' (x,y) = (\text{suma } x) y$

Diferencias

- Una toma un par, la otra toma un número
- Una retorna un número, la otra retorna *una función*
- La segunda forma es más expresiva
 - $\text{succF} = \text{suma } 1$ VS. $\text{succF}' x = \text{suma}' (1,x)$

Aplicación de funciones de orden

```
suma' :: (Int,Int) -> Int
suma' (x,y) = x+y

suma :: Int -> (Int -> Int)
suma x = f where f y = x+y
```

- Similitudes y diferencias, otra forma de verlo
(propuesta por Cristian Sottile)
 - Ambas expresan la suma de dos enteros
 - ¡Pero no de la misma manera!
 - `suma' (1,3) :: Int` -- Ambas expresan sumas
 - `(suma 1) 3 :: Int` -- Ambas expresan sumas
 - Pero una es más *expresiva* que la otra
 - `suma 1 :: Int->Int` -- Una puede expresar funciones...
 - no existe `e`.
 - `suma' e :: Int->Int` -- ...que la otra no

Curricación

- Definición de **curricación** (*currying*)
 - Correspondencia** uno a uno entre cada función que toma una tupla como argumento con una función que retorna una función intermedia que completa el trabajo
 - Pensando en una definición genérica
cada definición de la forma de **f'** se corresponde con una de la forma de **f**

$$\begin{aligned} f' &:: (A, B) \rightarrow C \\ f' (x, y) &= e \end{aligned}$$
$$\begin{aligned} f &:: A \rightarrow (B \rightarrow C) \\ f \ x &= g \text{ where } g \ y = e \end{aligned}$$

Currificación

Definición de currificación

Correspondencia uno a uno entre dos conjuntos de funciones

(Por cada f' hay una f , y viceversa)

$f' :: (A, B) \rightarrow C$

$f' (x, y) = e$

$f :: A \rightarrow (B \rightarrow C)$

$f x = g \text{ where } g y = e$

❏ Ejemplos de la *correspondencia* que es la currificación

`swap :: (a,b) -> (b,a)`

`swap (x,y) = (y,x)`

`riap :: a -> (b -> (b,a))`

`riap x = h`

`where h y = (y,x)`

Currificación

Definición de currificación

Correspondencia uno a uno entre dos conjuntos de funciones

(Por cada f' hay una f , y viceversa)

$f' :: (A, B) \rightarrow C$

$f' (x, y) = e$

$f :: A \rightarrow (B \rightarrow C)$

$f x = g \text{ where } g y = e$

■ Ejemplos de la *correspondencia* que es la currificación

`swap :: (a,b) -> (b,a)`

`swap (x,y) = (y,x)`

`riap :: a -> (b -> (b,a))`

`riap x = h`

`where h y = (y,x)`

`fst :: (a,b) -> a`

`fst (x,y) = x`

`const :: a -> (b -> a)`

`const x = f`

`where f y = x`

Curricación

Definición de currificación

Correspondencia uno a uno entre dos conjuntos de funciones

(Por cada f' hay una f , y viceversa)

$$\mathbf{f}' :: (\mathbf{A}, \mathbf{B}) \rightarrow \mathbf{C}$$
$$\mathbf{f}'(\mathbf{x}, \mathbf{y}) = \mathbf{e}$$
$$f :: A \rightarrow (B \rightarrow C)$$

f x = g where g y = e

📄 Ejemplos de la *correspondencia* que es la currificación

$$\begin{array}{c} \text{f'} \\ \text{swap} \end{array} :: \begin{array}{c} \text{A} \quad \text{B} \\ (\text{a}, \text{b}) \end{array} \rightarrow \begin{array}{c} \text{C} \\ (\text{b}, \text{a}) \end{array}$$

$$\begin{array}{c} \text{swap} \\ \text{f'} \end{array} (\text{x}, \text{y}) = \begin{array}{c} \text{e} \\ (\text{y}, \text{x}) \end{array}$$

```
fst :: (a,b) -> a
fst (x,y) = x
```

$$\begin{array}{c} \text{f} \quad \quad \quad \text{A} \quad \quad \quad \text{B} \quad \quad \quad \text{C} \\ \boxed{\text{riap}} :: \boxed{a} \rightarrow (\boxed{b} \rightarrow \boxed{(b, a)}) \\ \boxed{\text{riap}} \ \boxed{x} = \boxed{h} \\ \text{f} \quad \quad \quad \text{x} \quad \quad \quad \text{g} \\ \text{where } \boxed{h} \ \boxed{y} = \boxed{(y, x)} \\ \quad \quad \quad \text{g} \quad \text{y} \quad \quad \text{e} \end{array}$$

```
const :: a -> (b -> a)
const x = f
```

where $f_y = x$

Currificación

Definición de currificación

Correspondencia uno a uno entre dos conjuntos de funciones

(Por cada f' hay una f , y viceversa)

$f' :: (A, B) \rightarrow C$

$f' (x, y) = e$

$f :: A \rightarrow (B \rightarrow C)$

$f x = g \text{ where } g y = e$

❑ Ejemplos de la *correspondencia* que es la currificación

f' A B C
swap $:: (a, b) \rightarrow (b, a)$
swap $(x, y) = (y, x)$
 f' x y e

f A B C
riap $:: a \rightarrow (b \rightarrow (b, a))$
riap $x = h$
 f x g

where **h** $y = (y, x)$
 g y e

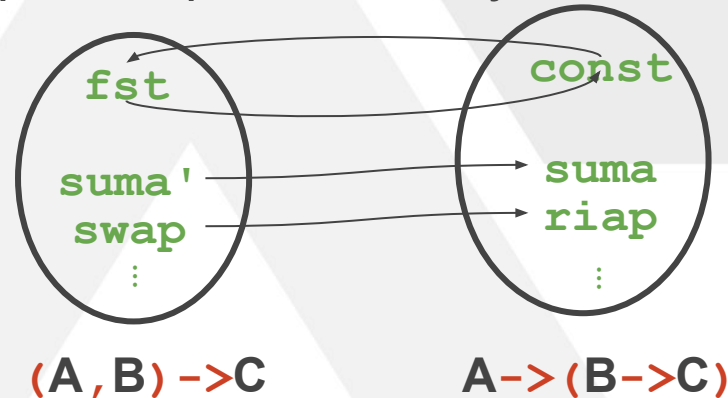
f' A B C
fst $:: (a, b) \rightarrow a$
fst $(x, y) = x$
 f' x y e

f A B C
const $:: a \rightarrow (b \rightarrow a)$
const $x = f$
 f x g

where **f** $y = x$
 g y e

Curricación

- Definición de currificación (*currying*)
 - Correspondencia** uno a uno entre cada función que toma una tupla como argumento con una función que retorna una función intermedia que completa el trabajo
 - Gráficamente



Curricación

- ❑ Sobre el nombre “curricación” (*currying*)
 - ❑ Es en honor a Haskell B. Curry
 - ❑ Aunque la idea la propuso Moses Schönfinkel
 - ❑ Pero schönfinkelización suena más complicado...
 - ❑ (y ni te cuento *schönfinkeling* para un inglés)
 - ❑ ...aunque hay quién lo sigue proponiendo

Haskell B. Curry



Haskell Brooks Curry

(12 de septiembre 1900 – 1 de septiembre 1982) fue un lógico y matemático estadounidense graduado de la Universidad de Harvard que alcanzó la fama por su trabajo en ***lógica combinatoria***. A pesar de que su trabajo se basó principalmente en un único artículo de Moses Schönfinkel, fue Curry el que hizo el desarrollo más importante.

También se lo conoce por la paradoja de Curry y por la *Correspondencia de Curry-Howard* (un vínculo profundo entre la lógica y la computación). Hay 3 lenguajes de programación nombrados en su honor, **Haskell**, **Brook** and **Curry**, así como el concepto de *currificación*, una técnica para aplicar funciones de orden superior.

Curricación

- ❑ Existe una correspondencia entre conjuntos

- ❑ ¿Se podrán definir funciones entre ellos?

- ❑ ¡Para eso armamos nuestro lenguaje funcional!

```
curry :: ((a,b) -> c) -> (a -> (b -> c))
```

```
curry ...
```

```
uncurry :: (a -> (b -> c)) -> ((a,b) -> c)
```

```
uncurry ...
```

- ❑ Con los elementos que veremos en esta clase podrán definirlas ustedes mismos

Currificación

■ Funciones de currificación

`curry` :: $((a,b) \rightarrow c) \rightarrow (a \rightarrow (b \rightarrow c))$

`uncurry` :: $(a \rightarrow (b \rightarrow c)) \rightarrow ((a,b) \rightarrow c)$

■ Se puede demostrar que

`curry (uncurry f)` = `f` para todo `f`

`uncurry (curry f')` = `f'` para todo `f'`

■ ¡Estamos expresando ideas complejas de programación con las herramientas que estudiamos!

Curricación

■ Funciones de currificación

curry :: ((a,b) -> c) -> (a -> (b -> c))

uncurry :: (a -> (b -> c)) -> ((a,b) -> c)

- De la función de tipo **a -> (b -> c)** se dice que está **currificada**

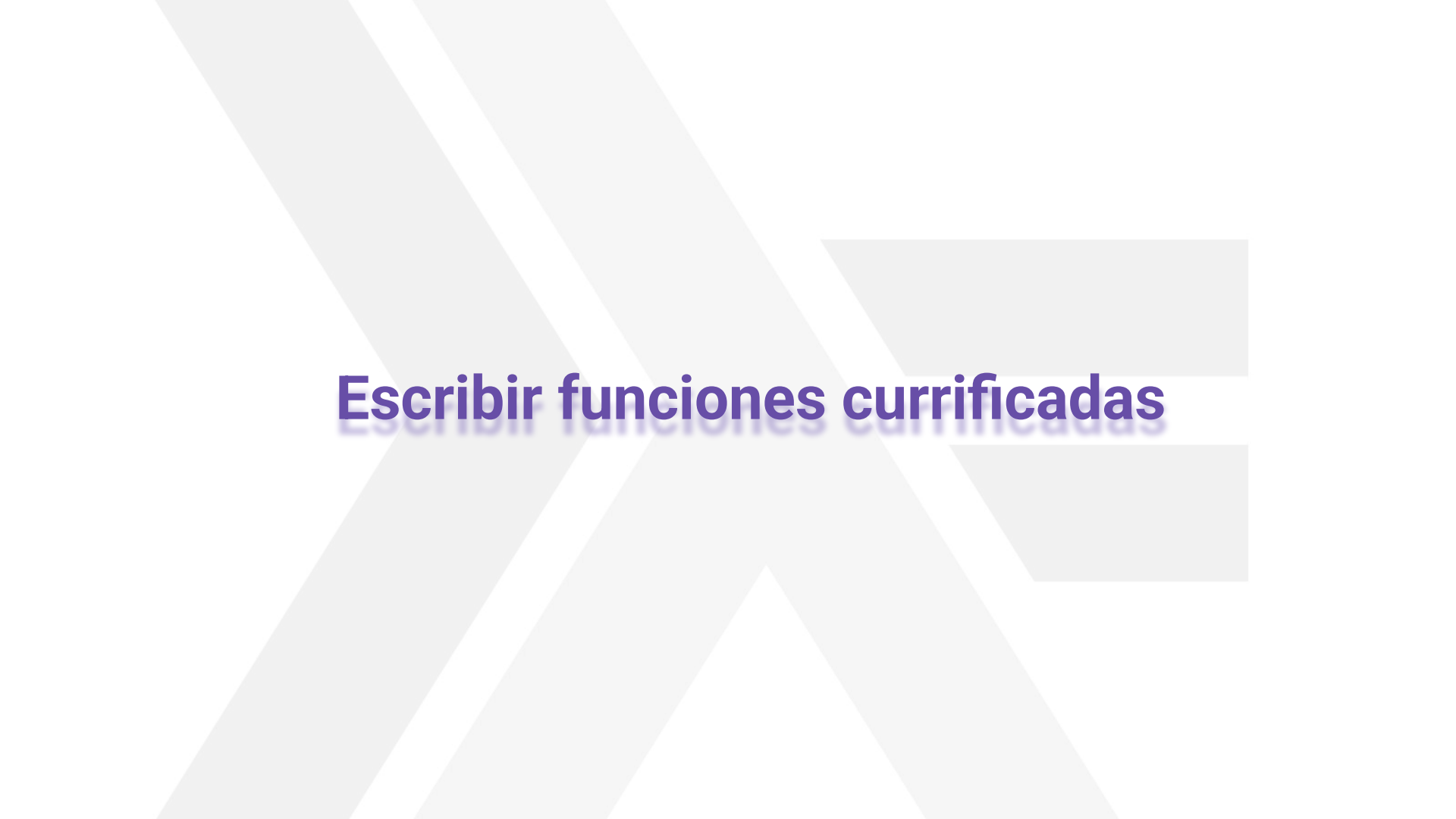
- En inglés, *curried*

- **f'** se *currifica* mediante **curry**, **(curry f') :: a -> (b -> c)**

- De la función de tipo **(a,b) -> c** se dice que está **descurrificada**

- En inglés, *uncurried* (en castellano, también **no currificada**)

- **f** se *descurrifica* con **uncurry**, **(uncurry f) :: (a,b) -> c**



Escribir funciones currificadas

Curricación - Sintaxis

- ❑ ¿Cómo podemos escribir una función curricada?
 - ❑ Hasta ahora, escribimos
$$\text{twice } f = g \quad \text{where } g \ x = f \ (f \ x)$$
 - ❑ Pero esto es incómodo...
 - ❑ ...como se puede comprobar con un ejemplo

Currificación - Sintaxis

- ❏ Considerar la función

```
suma5' :: (Int,Int,Int,Int,Int) -> Int
```

```
suma5' (x,y,z,v,w) = x+y+z+v+w
```

- ❏ ¿Cómo sería la versión currificada?

```
suma5 :: ...
```

```
suma5 ...
```

Currificación - Sintaxis

- ❏ Considerar la función

```
suma5' :: (Int,Int,Int,Int,Int) -> Int
```

```
suma5' (x,y,z,v,w) = x+y+z+v+w
```

- ❏ ¿Cómo sería la versión currificada?

```
suma5 ::      ->
```

```
suma5 ...
```

Debe tomar un
argumento solo y
devolver una función...

Currificación - Sintaxis

- ❏ Considerar la función

```
suma5' :: (Int,Int,Int,Int,Int) -> Int
```

```
suma5' (x,y,z,v,w) = x+y+z+v+w
```

- ❏ ¿Cómo sería la versión currificada?

```
suma5 :: Int->(    -> ... )
```

```
suma5 x = ...
```

Debe tomar un
argumento solo y
devolver una función...

Currificación - Sintaxis

- ❏ Considerar la función

```
suma5' :: (Int,Int,Int,Int,Int) -> Int
```

```
suma5' (x,y,z,v,w) = x+y+z+v+w
```

- ❏ ¿Cómo sería la versión currificada?

```
suma5 :: Int->(    -> ... )
```

```
suma5 x = ...
```

¿Qué nombre ponerle
a la función?

Currificación - Sintaxis

- ❏ Considerar la función

```
suma5' :: (Int,Int,Int,Int,Int) -> Int  
suma5' (x,y,z,v,w) = x+y+z+v+w
```

- ❏ ¿Cómo sería la versión currificada?

```
suma5 :: Int->(    -> ... )  
suma5 x = sum4  
  where sum4 ...
```

¿Qué nombre ponerle
a la función?

Currificación - Sintaxis

- ❏ Considerar la función

```
suma5' :: (Int,Int,Int,Int,Int) -> Int  
suma5' (x,y,z,v,w) = x+y+z+v+w
```

- ❏ ¿Cómo sería la versión currificada?

```
suma5 :: Int->(    -> ... )  
suma5 x = sum4  
  where sum4 ...
```

A su vez, esta función
debe tomar un
argumento solo y
devolver una función...

Currificación - Sintaxis

- ❏ Considerar la función

```
suma5' :: (Int,Int,Int,Int,Int) -> Int
suma5' (x,y,z,v,w) = x+y+z+v+w
```

- ❏ ¿Cómo sería la versión currificada?

```
suma5 :: Int->(Int->(    -> ... ))
suma5 x = sum4
  where sum4 y = ...
```

A su vez, esta función
debe tomar un
argumento solo y
devolver una función...

Currificación - Sintaxis

- ❑ Considerar la función

```
suma5' :: (Int,Int,Int,Int,Int) -> Int
suma5' (x,y,z,v,w) = x+y+z+v+w
```

- ❑ ¿Cómo sería la versión currificada?

```
suma5 :: Int->(Int-> ... )
suma5 x = sum4
  where sum4 y = sum3
    where sum3 ...
```

Curricación - Sintaxis

- ❏ Considerar la función

```
suma5' :: (Int,Int,Int,Int,Int) -> Int
suma5' (x,y,z,v,w) = x+y+z+v+w
```

- ❏ ¿Cómo sería la versión currificada?

```
suma5 :: Int->(Int->(Int-> ... ))
suma5 x = sum4
  where sum4 y = sum3
    where sum3 z = sum2
      where sum2 ...
```

Currificación - Sintaxis

- ❏ Considerar la función

```
suma5' :: (Int,Int,Int,Int,Int) -> Int
suma5' (x,y,z,v,w) = x+y+z+v+w
```

- ❏ ¿Cómo sería la versión currificada?

```
suma5 :: Int->(Int->(Int->(Int-> ... )))
suma5 x = sum4
  where sum4 y = sum3
    where sum3 z = sum2
      where sum2 v = sum1
        where sum1 ...
```


Curricación - Sintaxis

- ❏ Considerar la función

```
suma5' :: (Int,Int,Int,Int,Int) -> Int
suma5' (x,y,z,v,w) = x+y+z+v+w
```

- ❏ ¿Cómo sería la versión currificada?

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))
suma5 x = sum4
  where sum4 y = sum3
    where sum3 z = sum2
      where sum2 v = sum1
        where sum1 w = x+y+z+v+w
```

Curricación - Sintaxis

- ❑ Considerar la función

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))
```

```
suma5 x = sum4
```

```
  where sum4 y = sum3
```

```
    where sum3 z = sum2
```

```
      where sum2 v = sum1
```

```
        where sum1 w = x+y+z+v+w
```

- ❑ ¡Es horrible!
- ❑ ¿Podemos aprovechar el poder de la denotación?

Curricación - Sintaxis

- Analicemos las siguientes definiciones

doble x = x+x

vs.

dobleF = \x -> x+x

Curricación - Sintaxis

- Analicemos las siguientes definiciones

`doble x = x+x` vs. `dobleF = \x -> x+x`

- Se vé que `doble = \x->x+x` o sea `doble = dobleF`

Curricación - Sintaxis

- Analicemos las siguientes definiciones

`doble x = x+x` vs. `dobleF = \x -> x+x`

- Se vé que `doble = \x->x+x` o sea `doble = dobleF`

- En una definición, *la aplicación del lado izquierdo equivale a usar una función anónima (con un parámetro) del lado derecho*

Curricación - Sintaxis

- Analicemos las siguientes definiciones

`doble x = x+x` vs. `dobleF = \x -> x+x`

- Se vé que `doble = \x->x+x` o sea `doble = dobleF`

- En una definición, *la aplicación del lado izquierdo equivale a usar una función anónima (con un parámetro) del lado derecho*
- Abusando del vocabulario:
un argumento a la izquierda “pasa” como parámetro a la derecha

Curricación - Sintaxis

- Analicemos las siguientes definiciones

`doble x = x+x` vs. `dobleF = \x -> x+x`

- Se vé que `doble = \x->x+x` o sea `doble = dobleF`

- En una definición, *la aplicación del lado izquierdo equivale a usar una función anónima (con un parámetro) del lado derecho*

- Abusando del vocabulario:

un argumento a la izquierda “pasa” como parámetro a la derecha

`doble x = x+x`

“Pasar” un argumento:

`f x = e`

`f = \x -> e`

Curricación - Sintaxis

- Analicemos las siguientes definiciones

`doble x = x+x` vs. `dobleF = \x -> x+x`

- Se vé que `doble = \x->x+x` o sea `doble = dobleF`

- En una definición, *la aplicación del lado izquierdo equivale a usar una función anónima (con un parámetro) del lado derecho*

- Abusando del vocabulario:

un argumento a la izquierda “pasa” como parámetro a la derecha

`doble = \x->x+x`

“Pasar” un argumento:

`f x = e`

`f = \x -> e`

Curricación - Sintaxis

- ❑ ¿Cómo podemos escribir una función curricada?
- ❑ Veamos con el ejemplo de twice

```
twice f = g where g x = f (f x)
```

“Pasar” un argumento:

f x = e

f = \x -> e

Curricación - Sintaxis

❑ ¿Cómo podemos escribir una función curricada?

❑ Veamos con el ejemplo de twice

```
twice f = g where g x = f (f x)
```

❑ “Pasando” la **x** de **g**, podemos definir

```
twice f = g where g = \x -> f (f x)
```

“Pasar” un argumento:

f x = e

f = \x -> e

Curricación - Sintaxis

❑ ¿Cómo podemos escribir una función curricada?

❑ Veamos con el ejemplo de twice

```
twice f = g where g x = f (f x)
```

❑ “Pasando” la **x** de **g**, podemos definir

```
twice f = g where g = \x -> f (f x)
```

O sea `twice f = \x -> f (f x)`

“Pasar” un argumento:

```
f x = e
```

```
f = \x -> e
```

Curricación - Sintaxis

❑ ¿Cómo podemos escribir una función curricada?

❑ Veamos con el ejemplo de twice

```
twice f = g where g x = f (f x)
```

❑ “Pasando” la **x** de **g**, podemos definir

```
twice f = g where g = \x -> f (f x)
```

O sea `twice f = \x -> f (f x)`

❑ Y “pasando” la **x** para el otro lado

```
(twice f) x = f (f x)
```

“Pasar” un argumento:

```
f x = e
```

```
f = \x -> e
```

Currificación - Sintaxis

- ❑ ¿Cómo podemos escribir una función currificada?
- ❑ Formas equivalentes de definir a **twice**
 - ❑ `twice f = g where g x = f (f x)`
 - ❑ `twice f = \x -> f (f x)`
 - ❑ `twice = \f -> (\x -> f (f x))`
 - ❑ `(twice f) x = f (f x)`
- ❑ ¡¡Pero solo UNA de ellas!! ¿Cuál elegir?
- ❑ (En Haskell, una de estas formas puede traer problemas porque el sistema de tipos extendido no puede darle tipo)

Curricación - Sintaxis

- ❏ Volvamos a **suma5** y “pasemos” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))
```

```
suma5 x = sum4
```

```
  where sum4 y = sum3
```

```
    where sum3 z = sum2
```

```
      where sum2 v = sum1
```

```
        where sum1 w = x+y+z+v+w
```

Curricación - Sintaxis

- Volvamos a **suma5** y “pasemos” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))
```

```
suma5 x = sum4
```

```
  where sum4 y = sum3
```

```
    where sum3 z = sum2
```

```
      where sum2 v = sum1
```

```
        where sum1 w = x+y+z+v+w
```

“Pasar” un argumento:

```
f x = e
```

```
f = \x -> e
```

Curricación - Sintaxis

- Volvamos a **suma5** y “pasemos” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))
```

```
suma5 x = sum4
```

```
  where sum4 y = sum3
```

```
    where sum3 z = sum2
```

```
      where sum2 v = sum1
```

```
        where sum1 = \w->x+y+z+v+w
```

“Pasar” un argumento:

```
f x = e
```

```
f = \x -> e
```


Curricación - Sintaxis

- Volvamos a **suma5** y “pasemos” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))
```

```
suma5 x = sum4
```

```
  where sum4 y = sum3
```

```
    where sum3 z = sum2
```

```
      where sum2 v = sum1
```

```
        where sum1 = \w->x+y+z+v+w
```

Reemplazo de iguales
(sum1)

Curricación - Sintaxis

- Volvamos a **suma5** y “pasemos” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))
```

```
suma5 x = sum4
```

```
  where sum4 y = sum3
```

```
    where sum3 z = sum2
```

```
      where sum2 v = \w->x+y+z+v+w
```

Reemplazo de iguales
(sum1)



Curricación - Sintaxis

- Volvamos a **suma5** y “pasemos” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))
```

```
suma5 x = sum4
```

```
  where sum4 y = sum3
```

```
    where sum3 z = sum2
```

```
      where sum2 v = \w->x+y+z+v+w
```

“Pasar” un argumento:

```
f x = e
```

```
f = \x -> e
```

Curricación - Sintaxis

- Volvamos a **suma5** y “pasemos” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))
```

```
suma5 x = sum4
```

```
  where sum4 y = sum3
```

```
    where sum3 z = sum2
```

```
      where sum2 = \v->(\w->x+y+z+v+w)
```

“Pasar” un argumento:

```
f x = e
```

```
f = \x -> e
```

Curricación - Sintaxis

- Volvamos a **suma5** y “pasemos” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))
```

```
suma5 x = sum4
```

```
  where sum4 y = sum3
```

```
    where sum3 z = sum2
```

```
      where sum2 = \v->(\w->x+y+z+v+w)
```

Reemplazo de iguales
(sum2)

Curricación - Sintaxis

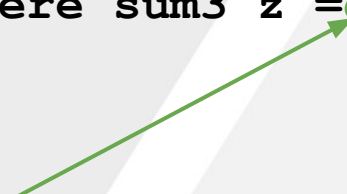
- Volvamos a **suma5** y “pasemos” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))
```

```
suma5 x = sum4
```

```
  where sum4 y = sum3
```

```
    where sum3 z = \v->(\w->x+y+z+v+w)
```



Reemplazo de iguales
(sum2)

Curricación - Sintaxis

- Volvamos a **suma5** y “pasemos” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))
```

```
suma5 x = sum4
```

```
  where sum4 y = sum3
```

```
    where sum3 z = \v->(\w->x+y+z+v+w)
```

“Pasar” un argumento:

f x = e

f = \x -> e

Curricación - Sintaxis

- Volvamos a **suma5** y “pasemos” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))
```

```
suma5 x = sum4
```

```
  where sum4 y = sum3
```

```
    where sum3 = \z->(\v->(\w->x+y+z+v+w))
```

“Pasar” un argumento:

f x = e

f = \x -> e

Curricación - Sintaxis

- ❏ Volvamos a **suma5** y “pasemos” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))
```

```
suma5 x = sum4
```

```
  where sum4 y = sum3
```

```
    where sum3 = \z->(\v->(\w->x+y+z+v+w))
```

Reemplazo de iguales
(sum3)

Curricación - Sintaxis

- Volvamos a **suma5** y “pasemos” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))
```

```
suma5 x = sum4
```

```
where sum4 y = \z->(\v->(\w->x+y+z+v+w))
```

Reemplazo de iguales
(sum3)

Curricación - Sintaxis

- Volvamos a **suma5** y “pasemos” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))
```

```
suma5 x = sum4
```

```
where sum4 y = \z->(\v->(\w->x+y+z+v+w))
```

“Pasar” un argumento:

```
f x = e
```

```
f = \x -> e
```

Curricación - Sintaxis

- Volvamos a **suma5** y “pasemos” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))
```

```
suma5 x = sum4
```

```
where sum4 = \y->(\z->(\v->(\w->x+y+z+v+w)))
```

“Pasar” un argumento:

$f\ x = e$

$f = \lambda x. e$

Curricación - Sintaxis

- Volvamos a **suma5** y “pasemos” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))
```

```
suma5 x = sum4
```

```
where sum4 = \y->(\z->(\v->(\w->x+y+z+v+w)))
```

Reemplazo de iguales
(sum4)

Curricación - Sintaxis

- Volvamos a **suma5** y “pasemos” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))
```

```
suma5 x = \y->(\z->(\v->(\w->x+y+z+v+w)))
```

Reemplazo de iguales
(sum4)

Curricación - Sintaxis

- ❑ Volvamos a **suma5** y “pasemos” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))
```

```
suma5 x = \y->(\z->(\v->(\w->x+y+z+v+w)))
```

- ❑ ¡La función es la misma!

- ❑ Podríamos “pasar” la x para el otro lado...

```
suma5 = \x->(\y->(\z->(\v->(\w->x+y+z+v+w))))
```

- ❑ ...¡o podríamos “pasar” los argumentos a la izquierda!

Curricación - Sintaxis

- Volvamos a **suma5** y sigamos “pasando” los argumentos

suma5 :: Int->(Int->(Int->(Int->(Int->Int))))

suma5 = \x->(\y->(\z->(\v->(\w->x+y+z+v+w))))

“Pasar” un argumento:

f x = **e**

f = \x -> e

Curricación - Sintaxis

- Volvamos a **suma5** y sigamos “pasando” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))
```

```
suma5 x = \y->(\z->(\v->(\w->x+y+z+v+w)))
```

“Pasar” un argumento:

```
f x = e
```

```
f = \x -> e
```

Curricación - Sintaxis

- Volvamos a **suma5** y sigamos “pasando” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))
```

```
suma5 x = \y->(\z->(\v->(\w->x+y+z+v+w)))
```

“Pasar” un argumento:

f x = e

f = \x -> e

Curricación - Sintaxis

- Volvamos a **suma5** y sigamos “pasando” los argumentos

suma5 :: Int->(Int->(Int->(Int->(Int->Int))))

(suma5 x) y = \z->(\v->(\w->x+y+z+v+w))

“Pasar” un argumento:

f x = e

f = \x -> e

Curricación - Sintaxis

- Volvamos a **suma5** y sigamos “pasando” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(suma5 x) y = \z->(\v->(\w->x+y+z+v+w))
```

“Pasar” un argumento:

f x = e

f = \x -> e

Curricación - Sintaxis

- Volvamos a **suma5** y sigamos “pasando” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
((suma5 x) y) z = \v->(\w->x+y+z+v+w)
```

“Pasar” un argumento:

f x = e

f = \x -> e

Curricación - Sintaxis

- Volvamos a **suma5** y sigamos “pasando” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
((suma5 x) y) z = \v->(\w->x+y+z+v+w)
```

“Pasar” un argumento:

f x = e

f = \x -> e

Curricación - Sintaxis

- Volvamos a **suma5** y sigamos “pasando” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v = \w->x+y+z+v+w
```

“Pasar” un argumento:

f x = e

f = \x -> e

Curricación - Sintaxis

- Volvamos a **suma5** y sigamos “pasando” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v = \w->x+y+z+v+w
```

“Pasar” un argumento:

f x = e

f = \x -> e

Curricación - Sintaxis

- Volvamos a **suma5** y sigamos “pasando” los argumentos

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```

“Pasar” un argumento:

f x = e

f = \x -> e

Curricación - Sintaxis

- Una definición equivalente de **suma5**

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```

- ¿Cómo se lee?

Curricación - Sintaxis

- Una definición equivalente de **suma5**

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```

- ¿Cómo se lee?

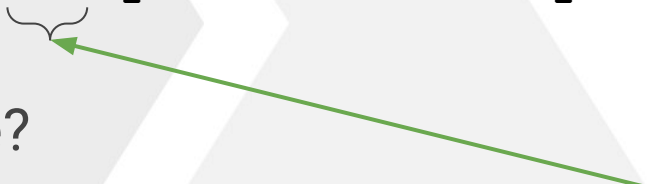
suma5 es una función

Curricación - Sintaxis

- Una definición equivalente de **suma5**

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```

- ¿Cómo se lee?



suma5 es una función *que toma un entero*

Curricación - Sintaxis

- Una definición equivalente de **suma5**

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```



- ¿Cómo se lee?

suma5 es una función que toma un entero y devuelve una función

Curricación - Sintaxis

- Una definición equivalente de **suma5**

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```

- ¿Cómo se lee?



suma5 es una función que toma un entero y devuelve una función *que toma un entero*

Curricación - Sintaxis

- Una definición equivalente de **suma5**

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))
```

```
((((suma5 x) y) z) v) w = x+y+z+v+w
```



- ¿Cómo se lee?

suma5 es una función que toma un entero y devuelve una función que toma un entero y *devuelve una función*

Curricación - Sintaxis

- Una definición equivalente de **suma5**

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```



- ¿Cómo se lee?

suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función *que toma un entero*

Curricación - Sintaxis

- Una definición equivalente de **suma5**

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```



- ¿Cómo se lee?

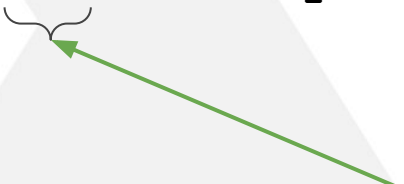
suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y *devuelve una función*

Curricación - Sintaxis

- Una definición equivalente de **suma5**

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```

- ¿Cómo se lee?



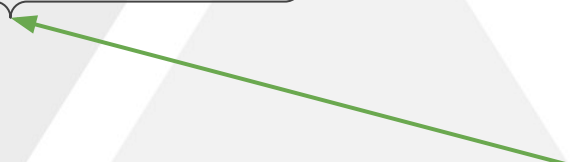
suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función *que toma un entero*

Curricación - Sintaxis

- Una definición equivalente de **suma5**

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))
```

```
((((suma5 x) y) z) v) w = x+y+z+v+w
```



- ¿Cómo se lee?

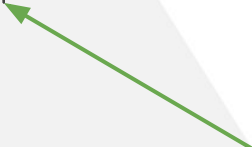
suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función

Curricación - Sintaxis

- Una definición equivalente de **suma5**

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```

- ¿Cómo se lee?



suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función *que toma un entero*

Curricación - Sintaxis

- Una definición equivalente de **suma5**

suma5 :: Int->(Int->(Int->(Int->(Int->Int))))

((((suma5 x) y) z) v) w = x+y+z+v+w

- ¿Cómo se lee?

suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve un entero

Curricación - Sintaxis

- Una definición equivalente de **suma5**

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```

- ¿Cómo se lee?

suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve un entero

Curricación - Sintaxis

- Una definición equivalente de **suma5**

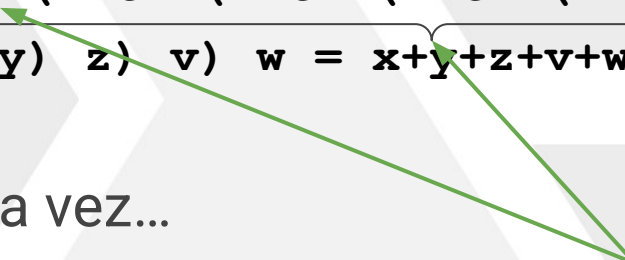
```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```

- ¿Cómo se lee? Otra vez...
- ...señalando el tipo

Currificación - Sintaxis

- Una definición equivalente de **suma5**

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```



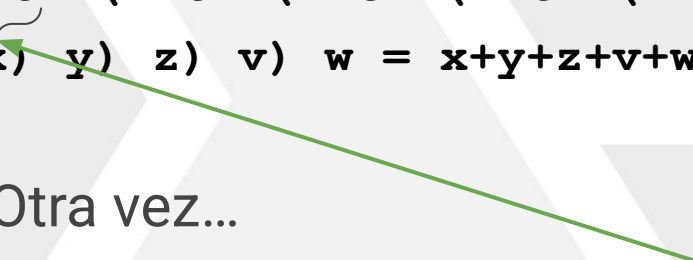
- ¿Cómo se lee? Otra vez...
- ...señalando el tipo

suma5 es una función

Curricación - Sintaxis

- Una definición equivalente de **suma5**

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```



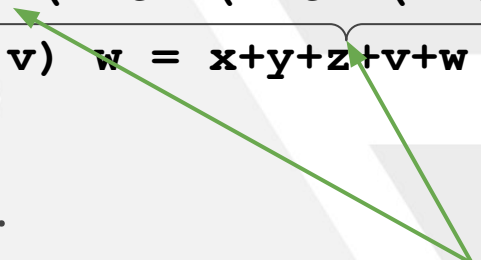
- ¿Cómo se lee? Otra vez...
- ...señalando el tipo

suma5 es una función *que toma un entero*

Curricación - Sintaxis

- Una definición equivalente de **suma5**

```
suma5 :: Int -> (Int -> (Int -> (Int -> (Int -> Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```



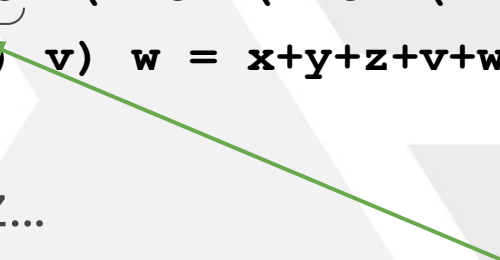
- ¿Cómo se lee? Otra vez...
- ...señalando el tipo

suma5 es una función que toma un entero y devuelve una función

Curricación - Sintaxis

- Una definición equivalente de **suma5**

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```



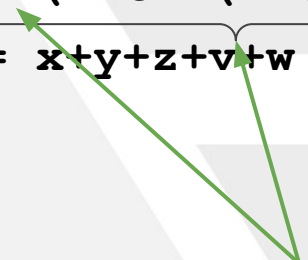
- ¿Cómo se lee? Otra vez...
- ...señalando el tipo

suma5 es una función que toma un entero y devuelve una función *que toma un entero*

Curricación - Sintaxis

- Una definición equivalente de **suma5**

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```



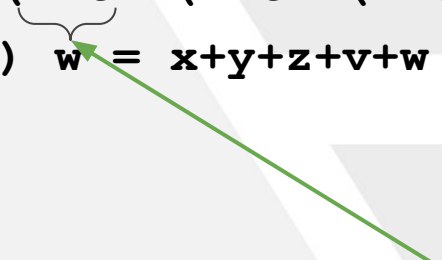
- ¿Cómo se lee? Otra vez...
- ...señalando el tipo

suma5 es una función que toma un entero y devuelve una función que toma un entero y *devuelve una función*

Curricación - Sintaxis

- Una definición equivalente de **suma5**

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```



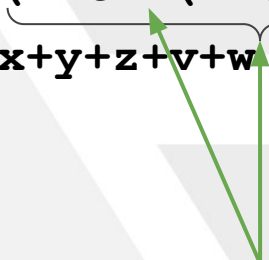
- ¿Cómo se lee? Otra vez...
- ...señalando el tipo

suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función *que toma un entero*

Curricación - Sintaxis

- Una definición equivalente de **suma5**

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```



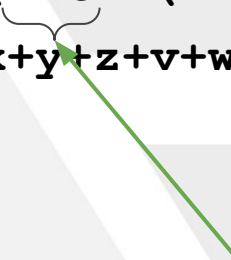
- ¿Cómo se lee? Otra vez...
- ...señalando el tipo

suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función

Curricación - Sintaxis

- Una definición equivalente de **suma5**

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```



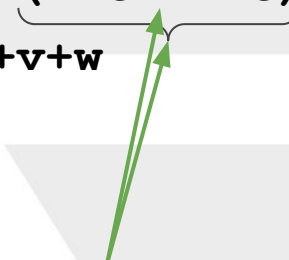
- ¿Cómo se lee? Otra vez...
- ...señalando el tipo

suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función *que toma un entero*

Curricación - Sintaxis

- Una definición equivalente de **suma5**

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```



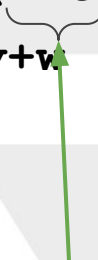
- ¿Cómo se lee? Otra vez...
- ...señalando el tipo

suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función

Currificación - Sintaxis

- Una definición equivalente de **suma5**

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```



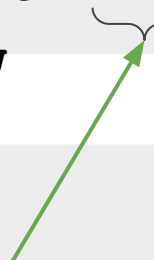
- ¿Cómo se lee? Otra vez...
- ...señalando el tipo

suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función *que toma un entero*

Curricación - Sintaxis

- Una definición equivalente de **suma5**

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int) )))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```



- ¿Cómo se lee? Otra vez...
- ...señalando el tipo

suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve un entero

Curricación - Sintaxis

- Una definición equivalente de **suma5**

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```

- ¿Cómo se lee?
- ¿No es molesto?
- ¿Cómo lo mejoramos?
- ¡Está lleno de paréntesis!

suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve un entero

Curricación - Sintaxis

- Consideremos esta expresión

10 - 3 - 2

- ¿Qué número denota? ¿5 o 9?

Curricación - Sintaxis

- Consideremos esta expresión

$$10 - 3 - 2$$

- ¿Qué número denota? ¿5 o 9?
 - ¿Es 7-2? ¿0 es 10-1?
- Si la resta es una operación binaria, ¿por qué hay 2 de ellas en la operación?

Curricación - Sintaxis

- Consideremos esta expresión

$$10 - 3 - 2$$

- ¿Qué número denota? ¿5 o 9?
 - ¿Es 7-2? ¿0 es 10-1?
- Si la resta es una operación binaria,
¿por qué hay 2 de ellas en la operación?
- ¿Cómo evitamos usar tantos paréntesis?
 - ¡Convenciones de notación!

Curricación - Sintaxis

- ❑ Convenciones de notación
 - ❑ La resta es “*asociativa a izquierda*”
 - ❑ O sea, $10 - 3 - 2 = (10 - 3) - 2$
 - ❑ $10 - 3 - 2 \neq 10 - (3 - 2)$
 - ❑ Si hay 2 símbolos seguidos iguales, se resuelve *primero el de la izquierda*
 - ❑ ¿Y “*asociativa a derecha*”?
 - ❑ ¿Y “*no asociativa*”?

Curricación - Sintaxis

- ❑ ¿Qué pasa con la aplicación de funciones?
 - ❑ Definimos que la aplicación es *“asociativa a izquierda”*
 - ❑ O sea, $(f\ x)\ y = f\ x\ y$
 - ❑ Recordar que el espacio es un símbolo...
 - ❑ ¡Los paréntesis a la izquierda no son necesarios!
 - ❑ Pero, f sigue siendo una función que toma x y devuelve una función que toma y y devuelve el resultado
 - ❑ ¡No es cierto que f tenga DOS argumentos!

Curricación - Sintaxis

- ❑ Convenciones de notación
 - ❑ La resta es “*asociativa a izquierda*”
 - ❑ O sea, $10 - 3 - 2 = (10 - 3) - 2$
 - ❑ $10 - 3 - 2 \neq 10 - (3 - 2)$
 - ❑ Si hay 2 símbolos seguidos iguales, se resuelve *primero el de la izquierda*
 - ❑ ¿Y “*asociativa a derecha*”?
 - ❑ ¿Y “*no asociativa*”?
 - ❑ ¡No puede haber dos símbolos seguidos sin paréntesis!

Curricación - Sintaxis

- ❑ ¿Y con el tipo de las funciones?
 - ❑ Definimos que la flecha es “asociativa a derecha”
 - ❑ O sea, $A \rightarrow (B \rightarrow C)$ es igual a $A \rightarrow B \rightarrow C$
 - ❑ ¡Los paréntesis a la derecha no son necesarios!
 - ❑ También sigue siendo una función que toma A y devuelve una función de B en C
 - ❑ Esta es una decisión coherente con la anterior
$$\begin{array}{ll} f :: A \rightarrow B \rightarrow C & \text{es} \quad f :: A \rightarrow (B \rightarrow C) \\ f \ x \ y = e & (f \ x) \ y = e \end{array}$$

Curricación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```

- Se pueden omitir
 - Los paréntesis a izquierda en las aplicaciones
 - Los paréntesis a la derecha de las funciones en el tipo

Currificación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

```
suma5 :: Int->Int->Int->Int->Int->Int
```

```
suma5 x y z v w = x+y+z+v+w
```

- Se pueden omitir
 - Los paréntesis a izquierda en las aplicaciones
 - Los paréntesis a la derecha de las funciones en el tipo

Curricación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

```
suma5 :: Int->Int->Int->Int->Int->Int
```

```
suma5 x y z v w = x+y+z+v+w
```

- Se pueden omitir
 - Los paréntesis a izquierda en las aplicaciones
 - Los paréntesis a la derecha de las funciones
- ¿Y esto cambió algo en la forma de leer?

Curricación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

```
suma5 :: Int->Int->Int->Int->Int->Int
```

```
suma5 x y z v w = x+y+z+v+w
```

- Se pueden omitir
 - Los paréntesis a izquierda en las aplicaciones
 - Los paréntesis a la derecha de las funciones
- ¿Y esto cambió algo en la forma de leer?
 - ¡NO!

Currificación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

```
suma5 :: Int->Int->Int->Int->Int->Int
```

```
suma5 x y z v w = x+y+z+v+w
```

- ¿Y esto cambió algo en la forma de leer?

- ¡NO!

suma5 es una *función* que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve un entero

Curricación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

```
suma5 :: Int->Int->Int->Int->Int->Int
```

```
suma5 x y z v w = x+y+z+v+w
```

- ¿Y esto cambió algo en la forma de leer?

- ¡NO!

suma5 es una función **que toma un entero** y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve un entero

Curricación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

```
suma5 :: Int->Int->Int->Int->Int->Int
```

```
suma5 x y z v w = x+y+z+v+w
```

- ¿Y esto cambió algo en la forma de leer?

- ¡NO!

suma5 es una función que toma un entero y **devuelve una función** que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve un entero

Currificación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

```
suma5 :: Int->Int->Int->Int->Int->Int
```

```
suma5 x y z v w = x+y+z+v+w
```

- ¿Y esto cambió algo en la forma de leer?

- ¡NO!

suma5 es una función que toma un entero y devuelve una función **que toma un entero** y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve un entero

Currificación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

```
suma5 :: Int->Int->Int->Int->Int->Int
```

```
suma5 x y z v w = x+y+z+v+w
```

- ¿Y esto cambió algo en la forma de leer?

- ¡NO!

suma5 es una función que toma un entero y devuelve una función que toma un entero y **devuelve una función** que toma un entero y devuelve una función que toma un entero y devuelve un entero

Currificación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

```
suma5 :: Int->Int->Int->Int->Int->Int
```

```
suma5 x y z v w = x+y+z+v+w
```

- ¿Y esto cambió algo en la forma de leer?

- ¡NO!

suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función **que toma un entero** y devuelve una función que toma un entero y devuelve un entero

Curricación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

```
suma5 :: Int->Int->Int->Int->Int->Int
```

```
suma5 x y z v w = x+y+z+v+w
```

- ¿Y esto cambió algo en la forma de leer?

- ¡NO!

suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y **devuelve una función** que toma un entero y devuelve una función que toma un entero y devuelve un entero

Curricación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

```
suma5 :: Int->Int->Int->Int->Int->Int
```

```
suma5 x y z v w = x+y+z+v+w
```

- ¿Y esto cambió algo en la forma de leer?

- ¡NO!

suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función **que toma un entero** y devuelve una función que toma un entero y devuelve un entero

Curricación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

```
suma5 :: Int->Int->Int->Int->Int->Int
```

```
suma5 x y z v w = x+y+z+v+w
```

- ¿Y esto cambió algo en la forma de leer?

- ¡NO!

suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y **devuelve una función** que toma un entero y devuelve un entero

Curricación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

```
suma5 :: Int->Int->Int->Int->Int->Int
```

```
suma5 x y z v w = x+y+z+v+w
```

- ¿Y esto cambió algo en la forma de leer?

- ¡NO!

suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función **que toma un entero** y devuelve un entero

Curricación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

suma5 :: Int->Int->Int->Int->Int->Int

suma5 x y z v w = x+y+z+v+w

- ¿Y esto cambió algo en la forma de leer?

- ¡NO!

suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y *devuelve un entero*

Curricación - Sintaxis

- Mejoremos la definición de usando **suma5** asociatividad

```
suma5 :: Int->Int->Int->Int->Int->Int
```

```
suma5 x y z v w = x+y+z+v+w
```

- ¿Y esto cambió algo en la forma de leer?

- ¡NO!

- ¿Se observan las diferentes funciones intermedias?

suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y *devuelve un entero*

Curricación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

```
suma5 :: Int->Int->Int->Int->Int->Int
```

```
suma5 x y z v w = x+y+z+v+w
```

- ¿Y esto cambió algo en la forma de leer?

- ¡NO!

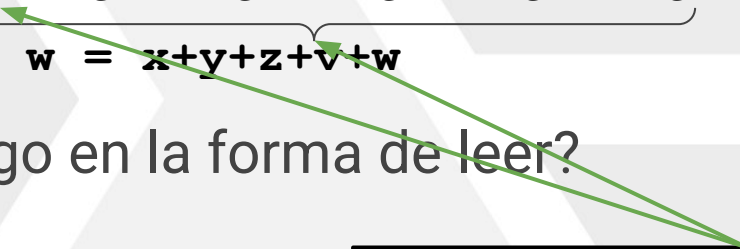
- ¿Se observan las diferentes funciones intermedias?

- ¡Y lo mismo en el tipo!

Curricación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

```
suma5 :: Int->Int->Int->Int->Int->Int
suma5 x y z v w = x+y+z+v+w
```



- ¿Y esto cambió algo en la forma de leer?

- ¡NO!
- ¿Se observan las diferentes funciones intermedias?
- ¡Y lo mismo en el tipo!

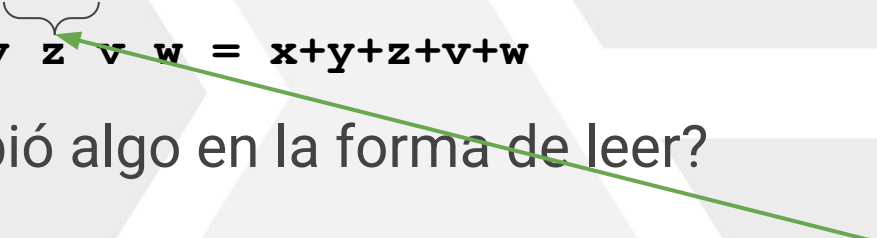
suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve un entero

Curricación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

suma5 :: Int->Int->Int->Int->Int->Int

suma5 x y z v w = x+y+z+v+w



- ¿Y esto cambió algo en la forma de leer?

- ¡NO!

- ¿Se observan las diferentes funciones intermedias?

- ¡Y lo mismo en el tipo!

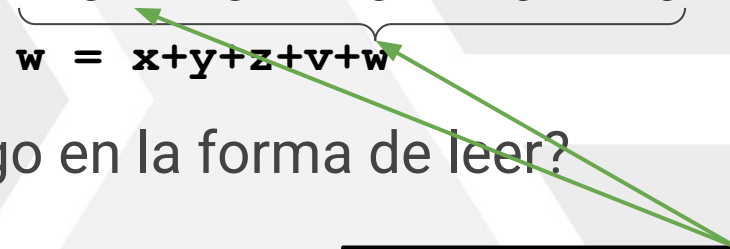
suma5 es una función **que toma un entero** y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve un entero

Curricación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

`suma5 :: Int->Int->Int->Int->Int->Int`

`suma5 x y z v w = x+y+z+v+w`



- ¿Y esto cambió algo en la forma de leer?

- ¡NO!
- ¿Se observan las diferentes funciones intermedias?
- ¡Y lo mismo en el tipo!

suma5 es una función que toma un entero y **devuelve una función** que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve un entero

Curricación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

suma5 :: Int->Int->Int->Int->Int->Int

suma5 x y z v w = x+y+z+v+w

- ¿Y esto cambió algo en la forma de leer?

- ¡NO!

- ¿Se observan las diferentes funciones intermedias?

- ¡Y lo mismo en el tipo!

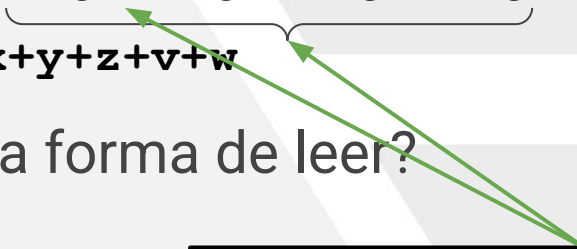
suma5 es una función que toma un entero y devuelve una función **que toma un entero** y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve un entero

Curricación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

`suma5 :: Int->Int->Int->Int->Int->Int`

`suma5 x y z v w = x+y+z+v+w`



- ¿Y esto cambió algo en la forma de leer?

- ¡NO!
- ¿Se observan las diferentes funciones intermedias?
- ¡Y lo mismo en el tipo!

suma5 es una función que toma un entero y devuelve una función que toma un entero y **devuelve una función** que toma un entero y devuelve una función que toma un entero y devuelve un entero

Curricación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

suma5 :: Int->Int->Int->Int->Int->Int

suma5 x y z v w = x+y+z+v+w



- ¿Y esto cambió algo en la forma de leer?

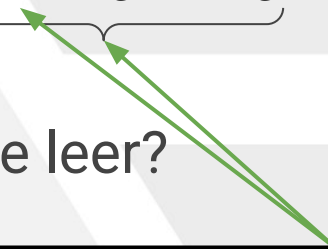
- ¡NO!
- ¿Se observan las diferentes funciones intermedias?
- ¡Y lo mismo en el tipo!

suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función **que toma un entero** y devuelve una función que toma un entero y devuelve un entero

Curricación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

```
suma5 :: Int->Int->Int->Int->Int->Int
suma5 x y z v w = x+y+z+v+w
```



- ¿Y esto cambió algo en la forma de leer?

- ¡NO!
- ¿Se observan las diferentes funciones intermedias?
- ¡Y lo mismo en el tipo!

suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y **devuelve una función** que toma un entero y devuelve una función que toma un entero y devuelve un entero

Curricación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

```
suma5 :: Int->Int->Int->Int->Int->Int
```

```
suma5 x y z v w = x+y+z+v+w
```



- ¿Y esto cambió algo en la forma de leer?

- ¡NO!
- ¿Se observan las diferentes funciones intermedias?
- ¡Y lo mismo en el tipo!

suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función **que toma un entero** y devuelve una función que toma un entero y devuelve un entero

Curricación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

```
suma5 :: Int->Int->Int->Int->Int->Int
```

```
suma5 x y z v w = x+y+z+v+w
```

- ¿Y esto cambió algo en la forma de leer?

- ¡NO!

- ¿Se observan las diferentes funciones intermedias?

- ¡Y lo mismo en el tipo!

suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y **devuelve una función** que toma un entero y devuelve un entero

Curricación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

```
suma5 :: Int->Int->Int->Int->Int->Int
```

```
suma5 x y z v w = x+y+z+v+w
```

- ¿Y esto cambió algo en la forma de leer?

- ¡NO!

- ¿Se observan las diferentes funciones intermedias?

- ¡Y lo mismo en el tipo!

suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función **que toma un entero** y devuelve un entero

Curricación - Sintaxis

- Mejoremos la definición de **suma5** usando asociatividad

```
suma5 :: Int->Int->Int->Int->Int->Int
```

```
suma5 x y z v w = x+y+z+v+w
```

- ¿Y esto cambió algo en la forma de leer?

- ¡NO!

- ¿Se observan las diferentes funciones intermedias?

- ¡Y lo mismo en el tipo!



suma5 es una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y devuelve una función que toma un entero y *devuelve un entero*

Curricación - Sintaxis

- Definición de **suma5** (currificada)

`suma5 :: Int->Int->Int->Int->Int->Int`

`suma5 x y z v w = x+y+z+v+w`

- Sigue siendo incómodo

- Comparar con

`suma5' :: (Int,Int,Int,Int,Int) ->Int`

`suma5' (x,y,z,v,w) = x+y+z+v+w`

suma5' es una función que toma cinco enteros y devuelve un entero

Currificación - Sintaxis

- Definición de **suma5** (currificada)

`suma5 :: Int->Int->Int->Int->Int->Int`

`suma5 x y z v w = x+y+z+v+w`

- Sigue siendo incómodo

- Comparar con

`suma5' :: (Int,Int,Int,Int,Int) -> Int`

`suma5' (x,y,z,v,w) = x+y+z+v+w`

`suma5'` es una función
que toma cinco enteros
y devuelve un entero

Currificación - Sintaxis

- Definición de **suma5** (currificada)

```
suma5 :: Int->Int->Int->Int->Int->Int
```

```
suma5 x y z v w = x+y+z+v+w
```

- Sigue siendo incómodo

- Comparar con

```
suma5' :: (Int,Int,Int,Int,Int) -> Int
```

```
suma5' (x,y,z,v,w) = x+y+z+v+w
```

suma5' es una función
que toma cinco enteros
y *devuelve un entero*

Currificación - Sintaxis

- Definición de **suma5** (currificada)

```
suma5 :: Int->Int->Int->Int->Int->Int
```

```
suma5 x y z v w = x+y+z+v+w
```

- Comparar con

```
suma5' :: (Int,Int,Int,Int,Int) ->Int
```

```
suma5' (x,y,z,v,w) = x+y+z+v+w
```

- ¿Con cuál nos quedamos?

Currificación - Sintaxis

- Definición de **suma5** (currificada)

```
suma5 :: Int->Int->Int->Int->Int->Int
```

```
suma5 x y z v w = x+y+z+v+w
```

- Comparar con la versión no currificada

```
suma5' :: (Int,Int,Int,Int,Int) ->Int
```

```
suma5' (x,y,z,v,w) = x+y+z+v+w
```

- ¿Con cuál nos quedamos?

- ¡A mitad de camino!

Currificación - Sintaxis

Definición de **suma5** (currificada)

suma5 :: Int->Int->Int->Int->Int->Int

suma5 x y z v w = x+y+z+v+w

suma5 es una función
que toma cinco enteros
y devuelve un entero

Curricación - Sintaxis

Definición de **suma5** (currificada)

suma5 :: Int->Int->Int->Int->Int->Int

suma5 x y z v w = x+y+z+v+w

suma5 es una función
que toma cinco enteros
y devuelve un entero

Curricación - Sintaxis

Definición de **suma5** (currificada)

suma5 :: Int->Int->Int->Int->Int->Int

suma5 x y z v w = x+y+z+v+w

suma5 es una función
que toma cinco enteros
y *devuelve un entero*

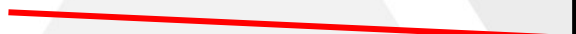
Currificación - Sintaxis

- Definición de **suma5** (currificada)

```
suma5 :: Int->Int->Int->Int->Int->Int
```

```
suma5 x y z v w = x+y+z+v+w
```

- ¡Pero no es correcto!



suma5 es una función
que toma cinco enteros
y devuelve un entero

Currificación - Sintaxis

- Definición de **suma5** (currificada)

```
suma5 :: Int->(Int->(Int->(Int->(Int->Int))))  
(((suma5 x) y) z) v) w = x+y+z+v+w
```

- ¡Pero no es correcto!

suma5 es una función
que toma cinco enteros
y devuelve un entero

Currificación - Sintaxis

- Definición de **suma5** (currificada)

suma5 :: Int->Int->Int->Int->Int->Int

suma5 x y z v w = x+y+z+v+w

- ¡Pero no es correcto!

- Leemos “*en francés*” a lo Libertad

- “¡Tratá de decir **suma5** ' pero pensar en **suma5**! ¡Dale, a ver, dale!”

suma5 ' es una función
que toma cinco enteros
y devuelve un entero

Currificación - Sintaxis

- Definición de **suma5** (currificada)

```
suma5 :: Int->Int->Int->Int->Int->Int
```

```
suma5 x y z v w = x+y+z+v+w
```

- ¡Pero no es correcto!

- Leemos “*en inglés*” a lo Libertad

- “¡Tratá de decir **suma5** ' pero pensar en **suma5**! ¡Dale, a ver, dale!”

- ¿Cuántos argumentos toma la función realmente?

suma5 es una función
que toma cinco enteros
y devuelve un entero



Ventajas de la currificación

Currificación - Ventajas

- ❑ Ventajas de la currificación
 - ❑ Mayor expresividad
 - ❑ Aplicación parcial
 - ❑ Modularidad
 - ❑ Para expresar mejor las ideas
 - ❑ Para tratamiento de código
 - ❑ Al inferir tipos
 - ❑ Al transformar programas

Curricación - Ventajas

- ❑ Ventajas: *Mayor expresividad*
 - ❑ Considerar la “derivada” de Análisis Matemático
 - ❑ En realidad, una aproximación algebraica...

```
derive :: (Float->Float) -> (Float->Float)
derive ...
```

Curricación - Ventajas

- ❑ Ventajas: *Mayor expresividad*
 - ❑ Considerar la “derivada” de Análisis Matemático
 - ❑ En realidad, una aproximación algebraica...

```
derive :: (Float->Float)->(Float->Float)  
derive f x = (f(x+h)-f x)/h where h = 0.0001
```

Curricación - Ventajas

- ❑ Ventajas: *Mayor expresividad*
 - ❑ Considerar la “derivada” de Análisis Matemático
 - ❑ En realidad, una aproximación algebraica...

```
derive :: (Float->Float)->(Float->Float)  
derive f x = (f(x+h)-f x)/h where h = 0.0001
```

- ❑ ¿Cuántas funciones se definieron?

Curricación - Ventajas

- ❑ Ventajas: *Mayor expresividad*
 - ❑ Considerar la “derivada” de Análisis Matemático
 - ❑ En realidad, una aproximación algebraica...

```
derive :: (Float->Float) -> (Float->Float)  
derive f x = (f(x+h) - f x) / h where h = 0.0001
```

- ❑ ¿Cuántas funciones se definieron? ¡DOS!
 - ❑ Una función que dado **f**, devuelve su *función* derivada
 - ❑ Una función que dado **f** y un punto **x**, devuelve la derivada de **f** en ese punto (un *número*)

Currificación - Ventajas

- ❑ Ventajas: *Mayor expresividad*
- ❑ Considerar la “derivada” de Análisis Matemático
 - ❑ En realidad, una aproximación algebraica...

```
derive :: (Float->Float)->(Float->Float)
derive f x = (f(x+h)-f x)/h where h = 0.0001
```

Estos paréntesis pueden ponerse para destacar la idea de que el resultado es una función...

Curricación - Ventajas

- ❑ Ventajas: *Mayor expresividad*
- ❑ Considerar la “derivada” de Análisis Matemático
 - ❑ En realidad, una aproximación algebraica...

```
derive :: (Float->Float)-> Float -> Float
derive f x = (f(x+h)-f x)/h where h = 0.0001
```

...o sacarse, para mostrar que el resultado final es un número

Curricación - Ventajas

- ❑ Ventajas: *Mayor expresividad*
- ❑ Considerar la “derivada” de Análisis Matemático
 - ❑ En realidad, una aproximación algebraica...

```
derive :: (Float->Float) -> (Float->Float)
derive f x = (f(x+h) - f x) / h where h = 0.0001
```

¡Estos paréntesis, en cambio,
son necesarios!

Curricación - Ventajas

- ❑ Ventajas: *Mayor expresividad*
 - ❑ Considerar la función “derivada” de Análisis Matemático
 - ❑ ¿Cuál sería la versión no currificada?

```
derive' :: (Float->Float,Float) -> Float
derive' ...
```
 - ❑ ¿Cuántas funciones se definirían?

Curricación - Ventajas

```
derive :: (Float->Float) -> Float -> Float
derive f x = (f(x+h)-f x)/h
              where h = 0.0001
```

- ❑ Ventajas: *Aplicación parcial*
- ❑ La función derive se puede aplicar a un solo argumento

```
derive f = \x->(f(x+h)-f x)/h where h = 0.0001
```

Curricación - Ventajas

```
derive :: (Float->Float) -> Float -> Float
derive f x = (f(x+h)-f x)/h
              where h = 0.0001
```

❑ Ventajas: *Aplicación parcial*

- ❑ La función `derive` se puede aplicar a un solo argumento

```
derive f = \x->(f(x+h)-f x)/h where h = 0.0001
```

- ❑ ¿Cómo sería la derivada n-ésima? (Usar `n` veces `derive`)

```
deriveN :: Int->(Float->Float)->(Float->Float)
deriveN ...
```

Curricación - Ventajas

```
derive :: (Float->Float) -> Float -> Float
derive f x = (f(x+h)-f x)/h
              where h = 0.0001
```

❑ Ventajas: *Aplicación parcial*

- ❑ La función `derive` se puede aplicar a un solo argumento

```
derive f = \x->(f(x+h)-f x)/h where h = 0.0001
```

- ❑ ¿Cómo sería la derivada n-ésima? (Usar `n` veces `derive`)

```
deriveN :: Int->(Float->Float)->(Float->Float)
```

```
deriveN 0 f = ...
```

```
deriveN n f = ...
```

Curricación - Ventajas

```
derive :: (Float->Float) -> Float -> Float
derive f x = (f(x+h)-f x)/h
              where h = 0.0001
```

❑ Ventajas: *Aplicación parcial*

- ❑ La función `derive` se puede aplicar a un solo argumento

```
derive f = \x->(f(x+h)-f x)/h where h = 0.0001
```

- ❑ ¿Cómo sería la derivada n-ésima? (Usar `n` veces `derive`)

```
deriveN :: Int->(Float->Float)->(Float->Float)
```

```
deriveN 0 f = f
```

```
deriveN n f = ...
```


Curricación - Ventajas

```
derive :: (Float->Float) -> Float -> Float
derive f x = (f(x+h)-f x)/h
              where h = 0.0001
```

❑ Ventajas: *Aplicación parcial*

- ❑ La función `derive` se puede aplicar a un solo argumento

```
derive f = \x->(f(x+h)-f x)/h where h = 0.0001
```

- ❑ ¿Cómo sería la derivada n-ésima? (Usar `n` veces `derive`)

```
deriveN :: Int->(Float->Float)->(Float->Float)
```

```
deriveN 0 f = f
```

```
deriveN n f = deriveN (n-1) (derive f)
```

Aplicación parcial de
`derive`

Curricación - Ventajas

```
derive :: (Float->Float) -> Float -> Float
derive f x = (f(x+h)-f x)/h
              where h = 0.0001
```

❑ Ventajas: *Aplicación parcial*

- ❑ La función `derive` se puede aplicar a un solo argumento

```
derive f = \x->(f(x+h)-f x)/h where h = 0.0001
```

- ❑ ¿Cómo sería la derivada n-ésima? (Usar `n` veces `derive`)

```
deriveN :: Int->(Float->Float)->Float->Float
```

```
deriveN 0 f = f
```

```
deriveN n f = deriveN (n-1) (derive f)
```

Aplicación parcial de
`derive`

```
derive' :: (Float->Float,Float) -> Float  
derive' ...
```

Curricación - Ventajas

❑ Ventajas: *Aplicación parcial*

❑ ¿Cómo sería la derivada n-ésima con **derive'**?

```
deriveN' :: (Int,Float->Float,Float) -> Float  
deriveN' ...
```

❑ ¡Es mucho más fácil con aplicación parcial!

❑ Y con aplicación parcial, se puede generalizar...

Curricación - Ventajas

- ❑ Ventajas: *Aplicación parcial*
- ❑ Escribir una función que aplique otra muchas veces

```
many :: Int -> (a->a) -> (a->a)
```

```
many ...
```

Curricación - Ventajas

- ❑ Ventajas: *Aplicación parcial*

- ❑ Escribir una función que aplique otra muchas veces

```
many :: Int -> (a->a) -> (a->a)
```

```
many 0 f x = ...
```

```
many n f x = ...
```

Curricación - Ventajas

- ❑ Ventajas: *Aplicación parcial*
- ❑ Escribir una función que aplique otra muchas veces

```
many :: Int -> (a->a) -> (a->a)
```

```
many 0 f x = x
```

```
many n f x = ...
```

Curricación - Ventajas

❑ Ventajas: *Aplicación parcial*

❑ Escribir una función que aplique otra muchas veces

```
many :: Int -> (a->a) -> (a->a)
```

```
many 0 f x = x
```

```
many n f x = f (many (n-1) f x)
```

Curricación - Ventajas

❑ Ventajas: *Aplicación parcial*

❑ Escribir una función que aplique otra muchas veces

```
many :: Int -> (a->a) -> a -> a
```

```
many 0 f x = x
```

```
many n f x = f (many (n-1) f x)
```


Curricación - Ventajas

- Ventajas: *Aplicación parcial*

- Escribir una función que aplique otra muchas veces

```
many :: Int -> ((a->a) -> (a->a))
```

```
many 0 f x = x
```

```
many n f x = f (many (n-1) f x)
```

Curricación - Ventajas

- ❑ Ventajas: *Aplicación parcial*
- ❑ Escribir una función que aplique otra muchas veces

```
many :: Int -> (a->a) -> (a->a)
many 0 f = \x -> x
many n f = \x -> f (many (n-1) f x)
```

Curricación - Ventajas

- ❑ Ventajas: *Aplicación parcial*

- ❑ Escribir una función que aplique otra muchas veces

`many :: Int -> (a->a) -> (a->a)`

`many 0 f x = x`

`many n f x = f (many (n-1) f x)`

- ❑ Se pueden expresar ideas que ya vimos

`?? = many 2`

`many 1 = ??`

para todo `n`,

`?? = many n derive`

- ❑ También se podrían usar para definir estas ideas...

Curricación - Ventajas

- ❑ Ventajas: *Aplicación parcial*

- ❑ Escribir una función que aplique otra muchas veces

`many :: Int -> (a->a) -> (a->a)`

`many 0 f x = x`

`many n f x = f (many (n-1) f x)`

- ❑ Se pueden expresar ideas que ya vimos

`twice = many 2`

`many 1 = ??`

para todo `n`,

`?? = many n derive`

- ❑ También se podrían usar para definir estas ideas...

Curricación - Ventajas

- ❑ Ventajas: *Aplicación parcial*

- ❑ Escribir una función que aplique otra muchas veces

```
many :: Int -> (a->a) -> (a->a)
```

```
many 0 f x = x
```

```
many n f x = f (many (n-1) f x)
```

- ❑ Se pueden expresar ideas que ya vimos

```
twice = many 2
```

```
many 1 = apply
```

```
para todo n,
```

```
?? = many n derive
```

- ❑ También se podrían usar para definir estas ideas...

Curricación - Ventajas

- ❑ Ventajas: *Aplicación parcial*

- ❑ Escribir una función que aplique otra muchas veces

```
many :: Int -> (a->a) -> (a->a)
```

```
many 0 f x = x
```

```
many n f x = f (many (n-1) f x)
```

- ❑ Se pueden expresar ideas que ya vimos

```
twice = many 2
```

```
many 1 = apply
```

```
para todo n, deriveN n = many n derive
```

- ❑ También se podrían usar para definir estas ideas...

Curricación - Ventajas

- Ventajas: *Aplicación parcial*

- La aplicación parcial permite cosas que parecen raras pero son poderosas

```
twice :: (a->a) -> a -> a
```

```
twice f x = f (f x)
```

- ¿Cuántos argumentos lleva **twice**?

Curricación - Ventajas

❑ Ventajas: *Aplicación parcial*

- ❑ La aplicación parcial permite cosas que parecen raras pero son poderosas

```
twice :: (a->a) -> a -> a
```

```
twice f x = f (f x)
```

- ❑ ¿Cuántos argumentos lleva **twice**?

twice

Ninguno

Curricación - Ventajas

■ Ventajas: *Aplicación parcial*

- La aplicación parcial permite cosas que parecen raras pero son poderosas

```
twice :: (a->a) -> a -> a
```

```
twice f x = f (f x)
```

- ¿Cuántos argumentos lleva **twice**?

twice

twice doble

Uno

Curricación - Ventajas

■ Ventajas: *Aplicación parcial*

- La aplicación parcial permite cosas que parecen raras pero son poderosas

```
twice :: (a->a) -> a -> a
```

```
twice f x = f (f x)
```

- ¿Cuántos argumentos lleva **twice**?

twice

twice doble

twice doble 2

Dos

Curricación - Ventajas

■ Ventajas: *Aplicación parcial*

- La aplicación parcial permite cosas que parecen raras pero son poderosas

```
twice :: (a->a) -> a -> a
```

```
twice f x = f (f x)
```

- ¿Cuántos argumentos lleva **twice**?

```
twice
```

```
twice doble
```

```
twice doble 2
```

```
twice twice doble 2
```

Tres

Curricación - Ventajas

■ Ventajas: *Aplicación parcial*

- La aplicación parcial permite cosas que parecen raras pero son poderosas

```
twice :: (a->a) -> a -> a
```

```
twice f x = f (f x)
```

- ¿Cuántos argumentos lleva **twice**?

twice

twice doble

twice doble 2

twice twice doble 2

twice twice twice doble 2

Cuatro

Curricación - Ventajas

■ Ventajas: *Aplicación parcial*

- La aplicación parcial permite cosas que parecen raras pero son poderosas

```
twice :: (a->a) -> a -> a
twice f x = f (f x)
```

- ¿Cuántos argumentos lleva **twice**?

twice

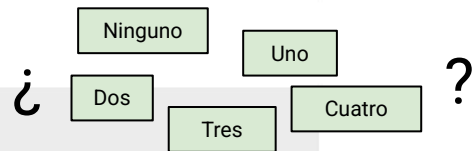
twice doble

twice doble 2

twice twice doble 2

twice twice twice doble 2

- ¡Todas estas expresiones tienen sentido!



Curricación - Ventajas

- ❑ Ventajas: *Aplicación parcial*

- ❑ ¿Cómo se explica **twice twice doble 2**?

Curricación - Ventajas

- ❑ Ventajas: *Aplicación parcial*
 - ❑ ¿Cómo se explica **(twice twice) doble 2?**

Curricación - Ventajas

❑ Ventajas: *Aplicación parcial*

❑ ¿Cómo se explica **(twice twice) doble 2?**

❑ Recordemos el tipado...

```
twice :: (a' -> a') -> a' -> a'
```

```
twice :: (a->a) -> (a->a)
```

```
twice twice ::
```


Curricación - Ventajas

❑ Ventajas: *Aplicación parcial*

❑ ¿Cómo se explica **(twice twice) doble 2**?

❑ Recordemos el tipado...

twice :: (a' -> a') -> a' -> a'

twice :: (a->a) -> (a->a)

twice twice :: (a->a) -> a -> a

a' <- a->a

Curricación - Ventajas

❑ Ventajas: *Aplicación parcial*

❑ ¿Cómo se explica **(twice twice) doble 2?**

❑ Recordemos el tipado...

twice :: (a' -> a') -> a' -> a'

twice :: (a->a) -> (a->a)

twice twice :: (a->a) -> a -> a

a' <- a->a

❑ ¡El **twice** de la izquierda es de 3er orden!

twice :: (**(a->a)****->****(a->a)****) ->****(a->a)****->****(a->a)****)**

a' a' a' a'

Curricación - Ventajas

❑ Ventajas: *Aplicación parcial*

❑ ¿Cómo se explica **twice twice doble 2**?

❑ Recordemos el tipado...

twice :: (a' -> a') -> a' -> a'

twice :: (a->a) -> (a->a)

twice twice :: (a->a) -> a -> a

a' <- a->a

❑ ¡El **twice** de la izquierda es de 3er orden!

twice :: $\underbrace{((a \rightarrow a) \rightarrow (a \rightarrow a))}_{a'} \rightarrow \underbrace{(a \rightarrow a)}_{a'} \rightarrow \underbrace{a}_{a'} \rightarrow a$

Curricación - Ventajas

❑ Ventajas: *Aplicación parcial*

❑ ¿Cómo se explica **(twice twice) doble 2?**

❑ Recordemos el tipado...

twice :: (a' -> a') -> a' -> a'

twice :: (a->a) -> (a->a)

twice twice :: (a->a) -> a -> a

a' <- a->a

❑ ¡El **twice** de la izquierda es de 3er orden!

twice :: (**(a->a)****->****(a->a)****) ->****(a->a)****->****(a->a)****)**

a' a' a' a'

Curricación - Ventajas

❑ Ventajas: *Aplicación parcial*

❑ ¿Cómo se explica `((twice twice) doble) 2?`

❑ Recordemos el tipado...

`twice :: (a' -> a') -> a' -> a'`

`twice :: (a->a) -> (a->a)`

`twice twice :: (a->a) -> a -> a`

`a' <- a->a`

❑ ¡El `twice` de la izquierda es de 3er orden!

`twice :: (`
 $\underbrace{\hspace{1.5cm}}$ $\underbrace{\hspace{1.5cm}}$ $\underbrace{\hspace{1.5cm}}$ $\underbrace{\hspace{1.5cm}}$
 `a'` `a'` `a'` `a'`
`) -> ((a->a) -> (a->a))`

Funciones “currificadas”

- ❑ ¿Reconocemos una función currificada solo por su tipo?

Funciones “currificadas”

- ❑ ¿Reconocemos una función currificada solo por su tipo?
- ❑ NO. Considerar la función “distancia al origen”

```
distancia :: (Float, Float) -> Float
```

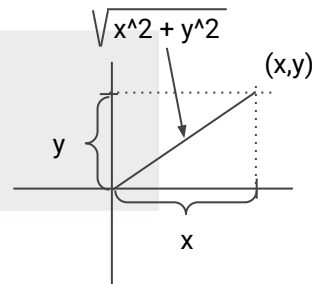
```
distancia ...
```

Funciones “currificadas”

- ❑ ¿Reconocemos una función currificada solo por su tipo?
- ❑ NO. Considerar la función “distancia al origen”

```
distancia :: (Float, Float) -> Float
```

```
distancia (x,y) = sqrt (x^2 + y^2)
```



Funciones “currificadas”

❑ ¿Reconocemos una función currificada solo por su tipo?

❑ NO. Considerar

```
distancia :: (Float, Float) -> Float
```

```
distancia (x,y) = sqrt (x^2 + y^2)
```

❑ ¿Tiene sentido separar el único argumento de **distancia**?

❑ Pero no devuelve una función... ¿y entonces?

Funciones “currificadas”

- ❑ ¿Reconocemos una función currificada solo por su tipo?
 - ❑ NO. Considerar

```
distancia :: (Float, Float) -> Float
distancia (x,y) = sqrt (x^2 + y^2)
```
 - ❑ ¿Tiene sentido separar el único argumento de **distancia**?
 - ❑ Pero no devuelve una función... ¿y entonces?
- ❑ La currificación puede ser *cuestión de interpretación*

Funciones “currificadas”

- ❑ La currrificación puede ser *cuestión de interpretación*
- ❑ ¿Y tiene sentido saber si algo está currrificado?
 - ❑ Considerar
$$\begin{aligned}\text{suma5''} &:: (\text{Int}, \text{Int}) \rightarrow \text{Int} \rightarrow (\text{Int}, \text{Int}) \rightarrow \text{Int} \\ \text{suma5''} \ (x, y) \ z \ (v, w) &= x + y + z + v + w\end{aligned}$$
 - ❑ ¿Podemos decir que está “*un poquito currrificada*”?
 - ❑ Mhh...



Resumen

Resumen

- ❏ Currificación
 - ❏ Aplicación de funciones de orden superior
 - ❏ Uso de Haskell para expresar ideas complejas
 - ❏ Importancia de la notación
 - ❏ Ventajas
 - ❏ Mucha mayor expresividad
 - ❏ Aplicación parcial
 - ❏ Modularidad